

Exploring AI Agent Systems and Orchestration

Directed Studies

Natalie Lambert

August 14, 2026

- Introduction
- Agent Architectures
- Workflow Coordination
- Context Management
- Measuring and Evaluating Agents
- Discussion and Future Work
- Conclusion

Introduction

Background and Motivation

- AI agent systems have emerged as a new way to integrate vanilla LLMs into composite systems
- In this directed studies, I treated LLMs as blackboxes, and examined how AI agents can be reasoned about from a systems perspective
- As LLM capabilities plateau, researchers examine ways that agent systems can surpass basic LLM performance

Agent Architectures

Single Agent Systems

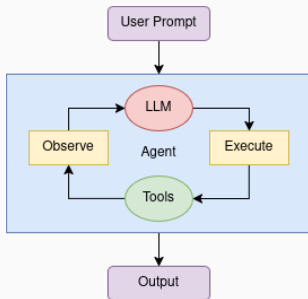


Figure 1: Most single-agent systems follow the Reason and Act (ReAct) paradigm.

- Comprised of an LLM, a tool call interface and an iterative harness
- Well-suited for tasks that allow for linear reasoning
- Most effective if using a frontier LLM

Multi-Agent Systems

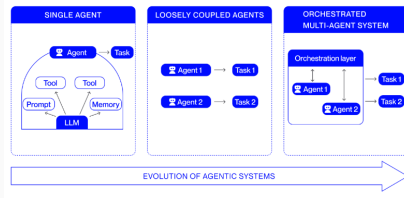


Figure 2: Agent systems have evolved to into more complicated interactions between single agents ([source](#)).

- A group of individual agents working in tandem on a task
- Can support non-linear and parallel reasoning sequences
- Depending on the workflow, inter-agent communication may be a bottleneck

Using Both: Cascade and Routing Strategies

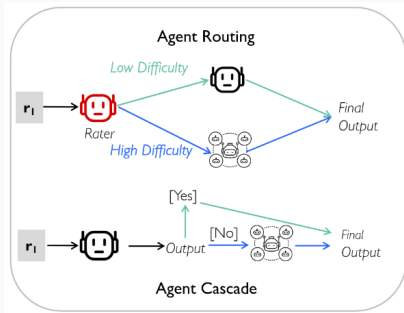


Figure 3: Schematic comparing cascade and routing between an SLM and LLM ([source](#)).

- So-called “hybrid” systems
- Routing: an LLM-as-judge determines a complexity score for a task, and assigns the task to a MAS (high complexity) or to a SAS (low complexity)
- Cascade: an SAS first attempts to solve a task; upon failure, the task is redirected to a MAS

Workflow Coordination

Independent Multi-Agent Systems

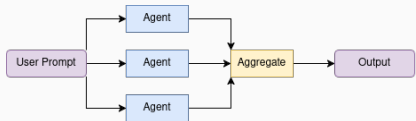


Figure 4: Three agents solve the same task in parallel, and all the outputs are aggregated for a final response.

- Multiple non-communicating agents solve tasks in parallel, after which all the outputs are aggregated
- Best used when it is useful to obtain multiple diverse answers, but they often require a subsequent deduplication step on the outputs
- Struggle with more data-driven or complicated task

Centralized Multi-Agent Systems

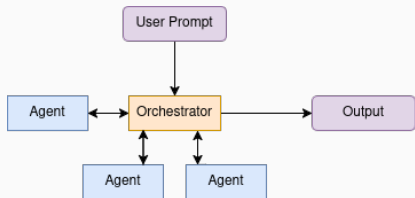


Figure 5: Three sub-agents are centrally controlled by an *orchestrator*, which resolves conflicts, deduplicates information and arbitrates between worker agents.

- Also called “hierarchical” or “layered” systems
- Top-down control where a higher-level orchestrator makes decisions, manages and delegates tasks to sub-agents
- Naturally suited for tasks that are easily decomposable

Decentralized Multi-Agent Systems

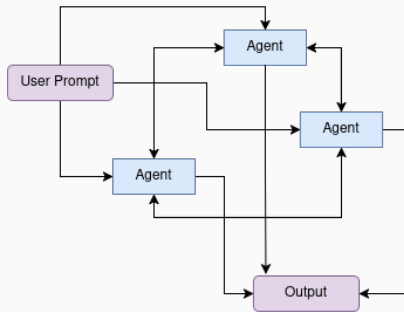


Figure 6: Three agents are organized in a peer-to-peer network, occasionally synchronizing.

- Distribute decision-making across a network of (mostly) independent agents (peer-to-peer)
- Peer-to-peer removes centralized bottlenecks but makes synchronization expensive
- Also highly suited to decomposable tasks, but inter-agent synchronization constraints must be relaxed to allow for somewhat infrequent output exchanges (e.g. debate scenario)

Context Management

Is Longer Context a Good Idea?

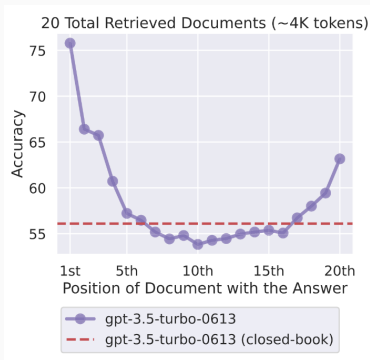


Figure 7: Longer context prevents agents from extracting necessary information that may be sandwiched in the middle ([source](#)).

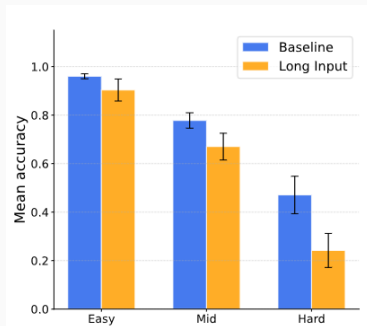


Figure 8: Irrelevant context build-up leads to accuracy drops and less self-verification ([source](#)).

Managing Context in Multi-Agent Systems

- Orchestrators in centralized multi-agent systems are often overwhelmed with context by sub-agents
- Passing only the minimal relevant context remains an issue
- Some papers propose using database-inspired transaction mechanisms and “context repos” in order to reliably track context accumulation over time, and to prune out old context
- Another approach is to dynamically limit the amount of context an orchestrator receives, to focus its reasoning on a single sub-agent at a time

Measuring and Evaluating Agents

Accuracy is Not the Only Metric You Need

- Benchmarks fail to do a holistic evaluation in AI agent capabilities, focusing mostly on accuracy scores
- Instead of pattern-matching, benchmarks often use LLM-as-judge to evaluate agent responses
- Cost, latency, efficacy, assurance and reliability are also important operational metrics
- Due to this discrepancy, agent systems tend to fail in real-world deployments
- Limited tracing practices prevents full introspection of agentic systems

Discussion and Future Work

Underexplored Aspects of Agent Systems

- Context management remains a key issue that affects how an agent system solves a task
- Agent observability is lacking or limited, so complex interactions between agents are not fully understood
- Setting constraints on agent systems is a good solution, but these design decisions are informed by a thorough understanding of agent heuristics and biases

Conclusion

Summary

- AI agents have shifted from being simple zero-shot interfaces to becoming autonomous systems that can attempt multi-step tasks with minimal human intervention
- However, these agents continue to lack reliability and optimal constraints that allow for bounded and more predictable behaviour
- This presentation showed the key architectural and orchestration strategies employed for single and multi-agent systems
- Context management and agent communication remain key challenges
- Better benchmarks and observability can help mitigate these issues

Questions?